Neuro-Evolved Heuristics for Variable Gapped Common Subsequence Identification

Marko Djukanović[1,5], Christian Blum[2], Aleksandar Kartelj[3], Sašo Džeroski[4], and Žiga Zebec[5]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iia.csic.es
[3] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[4] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[5] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

Abstract. This study addresses the Variable Gapped Longest Common Subsequence (VGLCS) problem, a variant of the classical longest common subsequence problem that incorporates additional gap constraints. The problem has applications in sequence alignment and time-series analysis. While the two-sequence version has been extensively studied using dynamic programming, its generalized form with multiple input sequences is typically tackled using beam search–based heuristics. However, existing hand-crafted heuristics often suffer from limited robustness.

To overcome this issue, we propose a learning-based approach for automatically designing more effective data-driven heuristics. The heuristics are modeled by a neural network with a predefined architecture, whose weights are optimized by a genetic algorithm in a neuro-evolutionary framework. The learning process alternates between evolutionary optimization of network weights and their evaluation within an iterative multi-source beam search procedure, a state-of-the-art method for the tackled problem. Importantly, the neural network does not directly construct solutions but instead learns to guide the search process, resulting in a neuro-evolved heuristic. Furthermore, we introduce an ensemble heuristic that combines the learned heuristics with the best-performing hand-crafted heuristic. The resulting hybrid approach, integrated into the iterative multi-source beam search framework, outperforms existing methods on both synthetic benchmark instances from the literature and the newly introduced real-world instances with data-driven gap constraints.

Keywords: Beam search; Neural networks; Neuro-Evolved heuristics; Gap constraints

1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [5] is a well-known combinatorial optimization problem with numerous applications in computational biology, particularly in the analysis of molecular sequences. Given an arbitrary set of input sequences $S = \{s_1, \ldots, s_m\}$ over an alphabet Σ, the goal is to determine the longest possible subsequence common to all $s_i \in S$. In recent decades, many practically motivated variants of the LCSP have been proposed by involving additional structural or biological constraints, defining the constrained, arc-preserving, repetition-free LCSP [8, 10], etc. In this work we study the *Variable Gapped* LCSP (VGLCSP), introduced in [13], theoretically examined in [1], and experimentally in [4]. This problem extends the standard LCSP by imposing flexible distance limits (gaps) between consecutive symbols in the resulting subsequence, allowing them to vary along the input sequences.

Formally, for each input sequence $s_i \in S$, a gap function $G_{s_i} \colon \{1, \ldots, |s_i|\} \mapsto \mathbb{N}$ defines the maximum distance allowed between the positions of appearances of consecutive characters in the resulting common subsequence for each $s_i, i = 1, \ldots, m$. Symbolically, if the characters of a solution s appear at the positions $i_i^1 < \ldots < i_i^{|s|} \leq |s_i|$ in s_i, the gap constraint is fulfilled iff $i_i^x - i_i^{x-1} \leq G_{s_i}[i_i^x] + 1$ holds for each $x = 2, \ldots, |s|$. A common subsequence s is considered feasible if the corresponding gap constraints hold across all $s_i \in S$. The problem aims at finding the longest such subsequence. The VGLCSP represents a flexible and realistic model of sequence alignment, which is particularly useful in comparing DNA or protein sequences with variable structural distances, but also in time series analysis, especially in settings where events are required to occur within specified temporal delays [9].

Although several exact dynamic programming approaches are proposed for two-sequence case ($m = 2$) scenario [12], the computational complexity of the problem becomes prohibitive for larger m; when m is arbitrarily large, the problem is NP-hard. To the best of our knowledge, there is no effective exact approach for general m, with just one approximate approach in the literature—an iterative beam search [4]—built on the state-space problem representation and the derived space of rooted nodes. Each such candidate node is explored via beam search in a two-step process: (i) the node refinement where non-rooted nodes are fixed to a (partial) solution constructed via a series of extensions towards a proven ancestor root node, and (ii) feasible expansions of the node in letter-by-letter manner selecting these from the associated subproblem assigned to the node. At each iterations, new candidate root nodes are detected and added in the queue of candidates. Both steps are performed by the beam search. Given that the methodology provides a scalable alternative to inefficient exact methods, this was the first approach providing experimental evaluations for arbitrary number of input sequences m. It relies on the use of several known hand-crafted (upper bound) LCS heuristics known from the literature as under-performers when m is large of Σ is small [5]. In the presence of gap constraints, this is even worsen. The second detected research gap concerns lack of real instances for the tackled problem as the synthetic instances are the only generated so far. Driven by

these issues, the first gap is filled by designing a learning iterative beam search, which combines hand-crafted search guidance and the learned guidance into a superior heuristic. The learned search guidance is produced as the outcome of a multi-layer neural network whose score is obtained in the forward propagation of the learned model. Weights of neural network model are evolved by a genetic algorithm while these neural network configurations are evaluated by the best heuristic method for the VGLCSP, the iterative beam search approach guided by the heuristic guidance induced from this network. The COP literature is known by utilizing similar (hybrid) methodology when learning heuristics, for example in [6] for the restricted LCS, [15] for the square LCS problem, or [19] for routing problems, as some notable example. The difference between these and our approach lies in the fact that learned heuristics are here further combined with other heuristics to design more superior ensemble heuristics. To address the second research gap, we have derived real problem instances by using (two) biologically-motivated benchmark sets from LCS literature, including data-driven gap constraints.

The paper is organized as follows. In Section 2, the iterative multi source beam search approach from the literature is roughly presented. Section 3 provides main details on the process of learning heuristic guidance, and the design of ensemble heuristic estimator for the tacked problem. Section 4 reports a rigorous experimental analysis. Finally, Section 5 concludes the paper highlighting several future research directions.

2 Iterative Multi-Source Beam Search for GLCS Problem

The first methodology for arbitrary m-case was introduced by Iterative multi-source beam search (IMSBS) in [4]. We only present it at certain details while interested readers are advised to consult the above-mentioned paper.

IMSBS is derived from the general *state graph* formulation for the VGLCSP. Each node represents a partial feasible subsequence, characterized by the vector of positions referring to a subproblem to be further solved, and the subsequence length. Symbolically, a state $v = (\mathbf{p}^{L,v}, l^v)$ is induced by a partial solution s^v if: (i) $p_i^{L,v} - 1$ is the smallest index such that s^v is a subsequence of the prefix string $s_i[1] \cdots s_i[p_i^{L,v} - 1]$; (ii) $l^v = |s^v|$; and (iii) all the gaps G_{s_i} are preserved with s^v. A directed arc $\alpha = (v_1 v_2)$, labeled $lett(\alpha) = a \in \Sigma$, exists between states v_1 and v_2 if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$. To expand a state v, all letters that occur in each (suffix) $S[\mathbf{p}^{L,v}]$ are considered. For each such letter a, the smallest positions $p_i^{L,a} \geq p_i^{L,v}$ of its appearance are identified fulfilling gap constraint at the position $p_i^{L,a}$ in s_i, i.e., $p_i^{L,a} - p_i^{L,v} \leq G_{s_i}(p_i^{L,a})$. The resulting child node—if not suboptimal—is given by $w = (p^{L,a} + 1, l^v + 1)$. Goal/target states correspond to non-extendable feasible subsequences.

Unlike the classical LCSP, the VGLCSP admits multiple (possibly exponentially many) root nodes as the starting position of a root node is constrained-free.

For example, in sequences $s_1 = \texttt{ATGGAAAA}$ and $s_2 = \texttt{ATCCAAAA}$ with $G_{s_1} = G_{s_2} = 1$, the (root) state $r = ((1,1),0)$ cannot reach any state with the position vector $(5,5)$ via feasible transitions, missing an opportunity to reach optimal subsequence $\texttt{AAA}$ if the search from r is initiated.

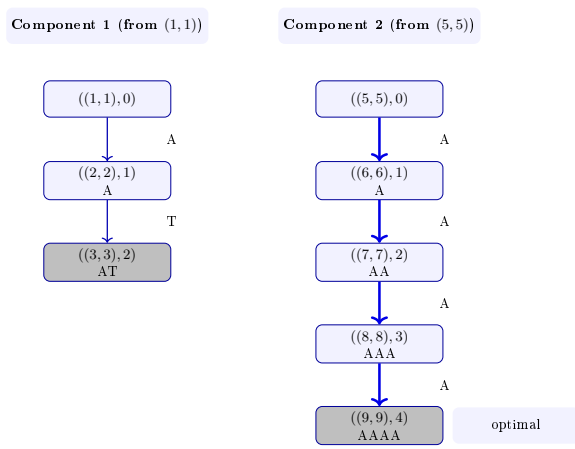

Fig. 1. State graph for $s_1 = \texttt{ATGGAAAA}$ and $s_2 = \texttt{ATCCAAAA}$ with $G_{s_1} = G_{s_2} = 1$. The state graph consists of two disconnected components. Starting from $(1,1)$ leads to a suboptimal solution, while starting from $(5,5)$ yields the optimal subsequence $\texttt{AAAA}$.

This motivates the development of IMSBS approach that works on a space of rooted nodes, which are dynamically generated over iterations and explored based on its priority. BS is a popular meta-heuristic approach [18] that performs in a breadth-first-search manner by expanding up to certain number of nodes at each level until the bottom is reached, returning the longest found path and its associated partial solution. The amount of nodes to further pursue is controlled by beam width (β)—the level of greediness—and a heuristic guidance (h) to select candidate nodes—as especially sensible algorithmic parameter. The heuristic used in [4] is given as $\texttt{UB}(v) = l^v + \sum_{a \in \Sigma} \min_{i=1,\ldots,m} |s_i[p_i^L, |s_i|]|_a$. For each letter $a \in \Sigma$, it first evaluates an upper bound on the number of its appearance in each residual sequence that corresponds to node v. These values are then sum up to generate a real upper bound value for the length of the LCS of the subproblem associated to node v.

In the subsequent part of the paper, we refer to the algorithm as $\texttt{IMSBS}$, returning an approximate solution s_{imsbs}.

3 Learning to Guide the Search

Neural networks (NNs) present a well-known machine learning model [7] that has shown its applicability in a wast majority of fields [3]. These models include

basic multi-layer perceptron towards more complex constitutional and recurrent neural networks of different architectures [11]. To keep it simple but efficient, we will consider only the fully-connected (multi-)layer NNs to derive an alternative heuristic to the known hand-crafted versions for guiding the search process of the IMSBS, see [4]. For doing so, the major issue is to properly parameterize the neural network, represented by set of weights assigned to its edges to induce a data-driven neural heuristic. It will be learned to predict a score for each node based on a set of extracted features extracted for associated partial solution and its subproblem.

In the context of classical neural networks, this learning process is performed by alternating between forward and backward propagation, where the later is used to weights adaptation w.r.t. error minimization (of a loss function) on a training dataset. The process of adapting weights is usually executed by employing iterative numerical procedures such as (stochastic) gradient descent [2] or the versions thereof. As we operate in the combinatorial space of an NP-hard problem, we do not expect the target values are known in advance, or there will be known to some extent but only for small-to-middle sized instances. This limits the application of gradient descent algorithms in training weights on such environments. We approach here differently by employing population-based meta-heuristic approach dealing with various NN configurations, which are evolved towards local optima. Fitness values of specific NN configurations are evaluated in a training instances by solving each of them using a robust (beam search) approach guided by the induced data-driven heuristic. As next we give details on the learning process.

Predefined Architecture of NN. The heuristic is represented by a *multi-layer perceptron* built of input of multiple neurons, output layer of one node, and a number of hidden layers with user-defined units per layer. The input layer received a set of numerical values associated to the features of subproblem to be processed. The output of NN refers to how much the considered subproblem is "promising". In each layer, to catch up non-linearity between input features, the following activation functions are tested: `hyperbolic tangent`, `ReLU`, and `sigmoid`. For each layer l, forward propagation is defined as $\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)})$, where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector, respectively, and ϕ denotes the activation function. The final $\mathbf{z}$-value represents the heuristic score. All weights and biases are flattened into a single vector $w = \texttt{flatten}(W)$, which forms the chromosome of a GA individual.
Fitness Evaluation of NN configurations. Given a candidate weight vector w, we perform: 1) Load of the weights into a FFNN architecture $\mathcal{N}$ of predefined architecture; 2) Executing the IMSBS on each instance from a training set $\mathcal{T}$ utilizing the neural heuristic to guide the search of IMSBS in the most sensitive forward iterations of BS; and 3) Recording size of the best sequence obtained for each instance. The fitness value of candidate vector w over the pre-defined NN architecture $\mathcal{N}$ is computed as the average solution quality over all training

instances. Formally, the learning problem is defined as:

$$\max_{w} \; \mathit{Fitness}(w) = \frac{1}{|\mathcal{T}|} \sum_{I \in \mathcal{T}} f(\texttt{IMSBS}(I, h_w)), \tag{1}$$

where h_w is the heuristic induced by neural network $\mathcal{N}(w)$, and $f(\cdot)$ is a problem-specific objective function—representing length of longest solution found by an IMSBS guided by h_w. Validation performance score is computed analogously on a separate dataset $\mathcal{V}$.

Feature Extraction. To guide the search process, each node is mapped to a fixed-length numerical feature vector capturing both the current state in construction phase and the structure of induced subproblem to solve. These features are designed to be instance-independent.

Let $S = \{s_1, \ldots, s_k\}$ denote the set of input sequences, and let $v = (\mathbf{p}^v, l^v)$ be a given node. The vector $\mathbf{p}^L = (\mathbf{p}_1^v, \ldots, \mathbf{p}_m^v)$ is first normalized by the corresponding sequence lengths $F^v = (F^v)_i \leftarrow \frac{\mathbf{p}_i^v}{|s_i|}, i = 1, \ldots, m$ so that progress in each sequence lies in $[0, 1]$ and becomes independent of absolute input sizes.

From this normalized position vector, four statistical descriptors are extracted: $\max(\mathbf{F})$, $\min(\mathbf{F})$, $\texttt{mean}(\mathbf{F})$, and standard deviation $\sigma(\mathbf{F})$. These quantities characterize, respectively, the most advanced and least advanced sequences, the average depth of the search, and the imbalance of progress among sequences. In addition, the length of currently constructed partial subsequence l^v is included as another feature. To capture the feasibility of future extensions, gap-related features are computed from the positions of remaining sequences. That is, for each v, we determine $G^{\min} = \min\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, $G^{\max} = \max\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, $G_i^{avg} = \texttt{mean}\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, and $G_i^{\texttt{std}} = \texttt{std}\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, where $G_1, \ldots, G_m$ are corresponding gap constraints associated to $s_1, \ldots, s_m$ respectively. These statistics for gap constrains summarize the remaining flexibility of the instance: large average or maximum gaps indicate greater freedom for future matches, whereas small minimum gaps and large variability typically signal tighter constraints and increased difficulty. Additional features involved are $|\Sigma|$ and the value of m (number of input sequences), providing global structural information about the instance and help the learning model adapt to varying problem scales. Note that we additionally probed with involving more features such as length of input sequences, however with a negative impact on solution quality.

Finally, all features are standardized to zero mean and unit variance prior to being passed to the learning component. The utilization of normalization is know to contribute decreased sensibility of learning process in the process of training ML models. Overall, the resulting representation combines information about (i) the current construction progress, (ii) the remaining gap-constraint landscape, and (iii) global instance properties, yielding 11 features. This compact yet expressive encoding enables effective learning-based guidance of the search for high-quality subsequences.

Optimize for high-quality configurations. We opt for a Genetic algorithm, in particular its popular version called the *Biased Random Key Genetic*

Algorithm (BRKGA) to solve the problem given in Eq. (1). The optimization process follows a population-based evolutionary scheme, described in Algorithm 1. Before we briefly explain the main workflow of the algorithm, first a few important notes. Each individual is initialized by sampling weights uniformly from a predefined interval $[-q, q]$ (q is fixed to 1).

Algorithm 1 GA-Based Learning of a BS Heuristic

Input: NN architecture $\mathcal{N}$, values of IMSBS parameters, values of BRKGA parameters, train set $\mathcal{T}$, validation set $\mathcal{V}$
Output: best weights for NN architecture
Initialize population P with random neural network weights
Evaluate all individuals on training set $\mathcal{T}$
Store best solution w^*
$start \leftarrow$ current time
while time limit $t_{\max}$ has not exceeded **do**
  Sort P by $Fitness(\cdot)$ value (descending)
  Initialize empty population P'
  Copy top n_{elites} individuals from P to P'
  for $i = 1$ to $n_{mutants}$ **do**
    Create random individual x
    Evaluate x on $\mathcal{T}$, add x to P'
    Update w^* if improved & validation score not decreased
  end for
  $P \leftarrow P[: |P| - n_{mutants}]$
  for $i = 1$ to $n_{offspring}$ **do**
    Select parents from the elite set $P[: n_{elite}]$ and the rest of individuals $P[n_{elite} :]$
    Generate offspring by crossover based on the inherited probability p_{bias}
    Evaluate offspring on $\mathcal{T}$
    Add offspring to P'
    Update w^* if improved & validation score not decreased
  end for
  $P \leftarrow P'$
end while
return w^*

Overall Learning Procedure. The learning process alternates between evolutionary optimization of network weights and evaluation through (iterative multi-source) Beam Search. Importantly, the neural network does not directly predict solutions; instead, it learns how to *guide* the search procedure. This places the method within the class of neuro-evolutionary hyper-heuristics.

Given are multi-layer perception architecture $\mathcal{N}$, parameters of IMSBS, and parameters of BRKGA to be involved in the fitness calculation of NN configurations. The procedure starts by generating an initial population P of individuals, representing random weights to embed into the predefined neural network architecture. All individuals are evaluated on the training set $\mathcal{T}$, yielding a fitness

value to reflect the quality of the corresponding heuristic/individual. The best-performing individual is stored as w^*. Note that the fitness evaluation for each individual has been obtained by employing IMSBS approach presented in Section 2, on each instance from $\mathcal{T}$. We here opt for the following parameters of IMSBS: $\beta = 500$, $beam_iters = 100$, while remaining parameters, therefore less sensible, are used from [4]. This provides a reasonable trade-off between solution quality in the learning process and the robustness of the training mechanism. The main iteration of the learning process executes the following steps. First, initial population of individuals P is sorted in descending order of their fitness values. The worst $n_{mutants}$ individuals by to the fitness values are removed from P, and the same amount of individuals is randomly generated and added to the newly generated population P' along with the n_{elite} (elite) individuals from P (which are copied to P'). The $n_{offspring}$ offsprings are generated by randomly selecting one parent from the pool of elite individuals, and other parent randomly from the remaining individuals in P. These two are subsequently combined for generating new offspring–inheriting the i-th component of first parent with probability $p_{bias} \geq 0.5$, otherwise the i-th component of the second parent with the probability $1 - p_{bias}$. Offsprings are passed to new generation P' after its fitness valuation. In case a new incumbent has reached on the set of training instances $\mathcal{T}$, the fitness is calculated on the set of validation instances $\mathcal{V}$. In particular, we decided to train in an early stopping fashion, terminating the process whenever the validation value decreases, as a criterion for detecting over-fitting. Otherwise, after generating all individuals, the population is updated ($P \leftarrow P'$). The process is repeated until the time limit $t_{\max}$ is reached—here fixed to 2 CPU hours. The algorithm returns the best solution w^*, corresponding to the neural network weights that induce the most effective search heuristic $h_w = \mathcal{N}(w)$ on the training set. As a reasonable optimizer, the BRKGA setting used for training takes the values $n_{pop} = 20, n_{elites} = 1, p_{bias} = 0.5, p_{mutants} = 7$. Visualization of the learning process is depicted in Fig. 2.

3.1 An ensemble approach

According to the preliminary results, the learning heuristics performs in overall better than the competitor hand-crafted heuristic from the literature. However, not for all cases, and especially on the instances with small m. In order to improve the results further, we employ a rank-based ensemble strategy that combines a learned heuristic with an analytical hand-crafted upper bound (UB) [4]. Specifically, each candidate node in beam search is evaluated using two criteria: a neural score obtained from a FFNN $\mathcal{N}(w^*)$, denoted $h_{w^*} = \mathcal{N}(w^*)$, and a problem-specific hand-crafted UB.

Instead of combining the raw scores directly, which may differ in scale and distribution, we adopt a rank aggregation approach. First, candidate nodes are sorted independently according to each criterion, producing two rankings. Each node is then assigned a rank based on its position in each sorted list. The final ranking score is computed as a weighted combination of the two ranks:

$$r(v) = \alpha \cdot r_{h_{w^*}}(v) + (1 - \alpha) \cdot r_{\mathrm{UB}}(v), \tag{2}$$

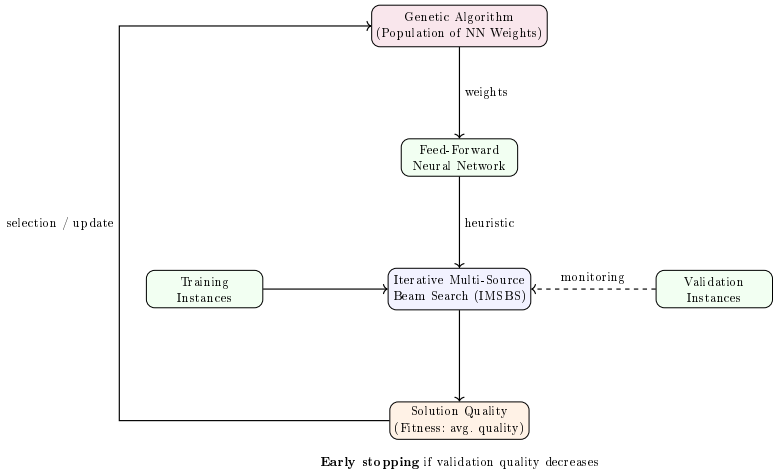

Fig. 2. Enhanced learning framework integrating a Genetic Algorithm with IMSBS. The GA evolves neural network weights, which define a heuristic used within the beam search. Training instances guide optimization, while validation instances monitor generalization. The feedback loop updates the population based on solution quality.

Nodes are then sorted in ascending order of $r(v)$, and the top candidates are retained according to the beam width. This rank-based ensemble heuristic provides a robust mechanism for integrating learned and analytical guidance, mitigating scale inconsistencies and improving search stability.

4 Experimental Evaluation

In this section we compare the performance of the two methods: (i) IMSBS: a baseline state-of-the-art iterative beam search strategy from [4], and (ii) LIMSBS-ensamble: a IMSBS guided by (2). All approaches are implemented in C++ and compiled via GCC compiler, evaluated on the VEGA HPC system, hosted at IZUM, Maribor, Slovenia. The cluster consists of 960 compute nodes equipped with the AMD EPYC 7H12 CPUs operating at 2.35 GHz. All experiments were conducted in a single-threaded setting.

Benchmark instances. For the evaluation purposes, two type of benchmark sets are used. The first set of synthetic instances is proposed in [4]. For each combination of $n \in \{50, 100, 200, 500\}$, $m \in \{2, 3, 5, 10\}$, and $|\Sigma| \in \{2, 4\}$, 10 random problem instances are randomly generated, a total of *320 problem instances*. The benchmark set is denoted as RANDOM. Apart from these, a set of biologically-motivated VGLCSP instances are introduced here.

In the context of real instances, we will be modeling context-dependent importance of positions of sequences that represents gap values. It aligns with sequence alignment scoring and weighted subsequence problems. Interpretation of these context relies on the fact that higher weights on positions refer to

stronger influence of these nucleotides in forming reliable connections with other (neighboring) acids in common subsequences. Stability ranking between pairs of nucleotides is (statistically) determined in the fundamental works on electrostatic interactions and structural influence [16, 17], used here in creating energy-aware LCS solutions, i.e., structure-aware variable-gap sequences (data-driven constraints):

`DINUC_WEIGHT` $= \{$'AA$'$: 1.00, 'TT$'$: 1.00, 'AT$'$: 0.88, 'TA$'$: 0.58, 'CA$'$: 1.45, 'TG$'$: 1.45, 'GT$'$: 1.44, 'AC$'$: 1.44, 'CT$'$: 1.28, 'AG$'$: 1.28, 'GA$'$: 1.30, 'TC$'$: 1.30, 'CG$'$: 2.17, 'GC$'$: 2.24, 'GG$'$: 1.84, 'CC$'$: 1.84$\}$ 'AA' and its score 1.00 means the stability of interactions between nucleotides `A` and `A` assign a value of 1.0. In essence, value of gap constraint at each position will be proportional to local thermodynamic stability estimated via nearest-neighbor interactions around the nucleotide placed at that position. *Dinucleotide-based structural model* is defined as a dinucleotide weight function: $f(s[i], s[i+1]) = -\Delta G(s[i], s[i+1])$, where ΔG denotes the nearest-neighbor free energy estimated by `DINUC_WEIGHT`.

To ensure consistency with the directional nature of the gapped LCS constraint, a *causal* positional contribution is derived. Each position k of a sequence s is associated only with the interaction formed with its immediate predecessor:

$$f_k^s = \begin{cases} f(s[k-1], s[k]), & k > 1, \\ 0, & k = 1. \end{cases}$$

Each position inherits structural influence from the bond linking it to the previous nucleotide. First position has no predecessor and is therefore assigned zero contribution. This definition enforces a left-to-right dependency structure, ensuring that all subsequent computations of influence and gap constraints depend only on previously observed positions. This is consistent with the feasibility condition in the gapped LCS problem, where admissibility of extending a subsequence depends on the last selected position.

The influence at position i is computed using only preceding positions: $I^s(i) = \sum_{k=\max(1,i-r)}^{i} e^{-\lambda(i-k)} \cdot f_k^s$. This formulation reflects the fact that, in the gapped LCS problem, feasibility of selecting position i depends on the previously selected position. Note that r is a parameter. The resulting influence values induce *directional structure* in the sequence: (i) regions preceded by strong interactions accumulate high influence, (ii) regions following weak or irregular segments exhibit lower influence.

In order to normalize these values, we employ the following formula:

$$\hat{I}_i^{(j)} = \frac{I_i^{(j)} - \min(I^{(j)})}{\max(I^{(j)}) - \min(I^{(j)}) + \epsilon} \in [0, 1]\,,$$

and gap constraints are obtained via a *direct mapping*: $w_i^{(j)} = w_{\min} + (w_{\max} - w_{\min}) \cdot \hat{I}_i^{(j)}$. In this context, high influence refers to large allowed gap, whilst low influence refers to restrictive gap.

This mapping reflects the idea that positions with strong structural connectivity can support longer-range dependencies allowing more flexible matching. Interval $[w_{\min}, w_{\max}]$ is parametrized, here fixed to 1 and 10, respectively.

The described instance generator is based on the following parameters: r as influence radius, λ as decay factor, $[w_{\min}, w_{\max}]$ as gap bounds. We additionally fixed $r = 10$, and $\lambda = 0.5$. The resulting benchmark enable systematic evaluation of multi-sequence gapped LCS algorithms under diverse and challenging conditions.

To demonstrate effectiveness of the designed approaches on real-world instances, we have used well-known LCS benchmark sets Rat and Virus [5]. For each input sequence in each problem instance, we have induced a respective data-driven gap constraint by the above methodology. We consider only the instances with $|\Sigma| = 4$ in both datasets, as the gap involvement for the instance with $|\Sigma| = 20$ induced a rather short solutions found by the approaches—therefore hardly relevant from the biological standpoint. The generated dataset, denoted by Real, comprises $2 \cdot 10 = 20$ problem instances. All sequences have a length of 600, and, as already mentioned, differentiate by m, ranging from 10 to 200.

All benchmark sets are freely available at XX. TODO: Revise

Tuning process. Imsbs parameters are tuned on benchmark set Random by employing `irace` [14], with an allowed training budget of 5000 runs (each run is limited to an execution time of 30 minutes) producing the following Imsbs configuration: $\beta = 3000$, $h = \text{UB}$, and $\#imbs_iters = 5000$. One instance from each group has been used for the training, thus 32 problem instances in overall.

All other (less sensible) parameters are set to the values reported in [4]. Concerning the learning Limsbs-ensemble approach, it inherits the same setting form Imsbs, to be completely fair with the comparison. For the learning process, we experimented with different architectures for NN: 2-layer structures with 5 and 10 neurons per layer, and 3-layer configurations with 5 and 10 neurons.

In order to reduce time for tuning different neural network architecture, we allow setting up the same activation function at each layer, selecting among three (most) known: $\{\texttt{tanh}, \texttt{ReLU}, \texttt{sigmoid}\}$. To measure the sensibility analysis w.r.t. set of (11) involved features, we have split them into three groups: (i) Feature configuration 1 F_1 involves 4 features extracted from input sequences, 4 features extracted from gap constraints, and length of partial solution; (ii) When adding $|\Sigma|$ to F_1, we get feature configuration F_2; (iii) When adding m to F_2, we obtain the feature configuration F_3.

After exhaustive training evaluations, we opt to chose the following configurations for the architecture of neural networks, one per different m in the case of benchmark set Random:

- $m = 2$: units at each layer of FFNN is (10, 5, 5); activation function is `ReLu`; and the feature set F_2;
- $m = 3$: units at each layer of FFNN set to (10, 10, 5); activation function is set to `ReLu`; and the feature set to F_2;
- $m = 5$: units at each layer of FFNN set to (10, 5, 5), activation function set to `sigmoid`, and the feature set to F_1;
- $m = 10$: units at each layer of FFNN set to (10, 5, 5); activation function set to `ReLu`, and the feature set to F_2.

Table 1. Comparison of objective values between IMSBS and LIMSBS-ensemble on benchmark set RANDOM. Best results per instance category are displayed in bold.

m	n	$\|\Sigma\|$	LIMSBS-ensemble	IMSBS
2	50	2	37.3	**37.7**
2	50	4	**30.1**	**30.1**
2	100	2	72.4	**72.9**
2	100	4	**62.0**	61.9
2	200	2	**142.5**	139.3
2	200	4	**125.9**	125.5
2	500	2	**305.9**	282.1
2	500	4	**305.5**	303.0
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	100	2	**58.5**	**58.5**
3	100	4	**48.7**	48.5
3	200	2	**106.7**	106.4
3	200	4	**97.9**	97.8
3	500	2	**154.9**	150.6
3	500	4	**237.6**	236.0
Avg.			**115.0**	112.8

m	n	$\|\Sigma\|$	LIMSBS-ensemble	IMSBS
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	100	2	32.6	**32.9**
5	100	4	28.0	**28.1**
5	200	2	**45.5**	42.1
5	200	4	**41.8**	38.8
5	500	2	**45.7**	39.2
5	500	4	**58.0**	45.6
10	50	2	**11.7**	**11.7**
10	50	4	7.3	**7.5**
10	100	2	**14.3**	12.0
10	100	4	**9.6**	8.9
10	200	2	**14.3**	12.8
10	200	4	**9.4**	8.2
10	500	2	**14.3**	13.2
10	500	4	**10.0**	8.6
Avg.			**23.6**	21.6

The parameter $\alpha = 0.45$ in our ensemble approach except for $m = 3$ where $\alpha = 0.5$ as the results of grid search. We employed 32 instances for the training instances $\mathcal{T}$ and the same amount for the validation instances $\mathcal{V}$; one instance is selected randomly per each (32) group from benchmark set RANDOM. Thus, eight instances are used per each learning process—split by different m—for training and validation purposes.

For the benchmark set REAL, the employed architecture of NN is the same utilized for the benchmark set RANDOM and the largest $m = 10$ case. This is because we wanted to test the robustness of the learned heuristic on other than random instance distributions with a large number of sequences (m). Note that all 20 instances have 10 or more sequences. The parameters of IMSBS follows those settings from the benchmark set RANDOM.

4.1 Numerical results on benchmark set RANDOM

These results are presented in Table 1 which consists of two blocks: the first gives an instance group description, the next block is reserved for the results of LIMSBS-ensemble and IMSBS approaches, reporting the average results of each group (over 10 instances).

Additionally, we evaluated the difference in solution quality between LIMSBS-ensemble and IMSBS using the Wilcoxon signed-rank test on paired instances, reported in Table 2.

The test reveals a statistically significant difference in favor of LIMSBS-ensemble, indicating that the learned approach achieves superior performance overall. Out of 32 instances, LIMSBS-ensemble outperforms IMSBS in 20 cases, while 7 instances result in ties and only 5 favor IMSBS.

Table 2. Statistical comparison between LIMSBS-ensemble and IMSBS using the Wilcoxon signed-rank test (one-sided).

Comparison	Wins / Ties / Losses	p-value	Conclusion
All instances ($m = 2, 3, 5, 10$)	20 / 7 / 5	0.009	LIMSBS significantly better
Small instances ($m = 2, 3$)	10 / 4 / 2	< 0.01	Significant difference
Large instances ($m = 5, 10$)	10 / 3 / 3	< 0.05	LIMSBS significantly better

4.2 Numerical results on benchmark set REAL

The results are reported in Table 3, organized in a similar manner as Table 1 (with additional columns, reporting the runtimes of each approach per instance ($t[s]$) and lengths of the best LCS from the literature [5] (LCS*)). The following conclusions are drawn.

Table 3. Comparison of LIMSBS-ensemble and IMSBS on benchmark set REAL.

RAT instances

m	LIMSBS-ensemble	$t[s]$	IMSBS	$t[s]$	LCS*
10	**28**	43.44	25	30.65	209
15	**11**	23.39	**11**	24.26	189
20	**10**	11.31	9	49.36	174
25	**9**	17.53	7	29.62	173
40	**5**	18.18	4	48.64	154
60	**3**	22.62	1	45.22	154
80	**1**	22.52	**1**	53.02	144
100	**1**	25.71	**1**	44.30	139
150	**1**	39.51	**1**	66.57	131
200	**1**	39.82	**1**	62.54	126

VIRUS instances

m	LIMSBS-ensemble	$t[s]$	IMSBS	$t[s]$	LCS*
10	**87**	134.51	36	43.14	228
15	**19**	27.08	**19**	36.52	206
20	**11**	16.31	10	26.92	194
25	**13**	16.66	9	30.40	196
40	**7**	20.39	5	32.12	174
60	**5**	20.26	4	34.38	168
80	**4**	22.80	3	40.64	163
100	**3**	35.27	**3**	43.25	160
150	**4**	27.71	3	31.80	157
200	**1**	32.06	**1**	48.59	156

- Overall, LIMSBS-ensemble outperforms IMSBS in 15 out of 20 instances, while 5 instances result in ties and none favor IMSBS. This demonstrates a consistent dominance of LIMSBS-ensemble across the tested dataset.
- In terms of solution quality, LIMSBS-ensemble achieves an average value of 11.8, compared to 9.9 for IMSBS, corresponding to an average relative improvement of approximately 19%. The median quality values are 5 (LIMSBS-ensemble) and 4 (IMSBS), confirming that improvements are consistent and not driven by outliers.
- The advantage of LIMSBS-ensemble is particularly pronounced on more difficult instances (e.g., virus cases with higher variability), where substantial gains are observed (e.g., 87 vs. 36). In contrast, ties typically occur on simpler instances with very low objective values.
- Regarding computational time, IMSBS is generally faster, but achieving an average runtime of 36.9 seconds compared to 32.6 seconds for the ensemble LIMSBS-ensemble. However, this difference is not consistent across all instances, and in several cases LIMSBS-ensemble is competitive or even faster.
- When comparing the lengths of best (constrained-free) LCS from the literature, it is obvious that gap constraints, and specifically those included in this work, affecting final solutions significantly. For the instances with $m \geq 60$ in many cases found solutions are even trivial indicating the strong influence

of a huge number of gap constraints in restricting search space of common subsequences fulfilling these.

The results demonstrate that the LIMSBS-ensemble provides a substantial and consistent improvement in solution quality over IMSBS on real-world instances. This makes it a preferable choice when solution quality is the main objective.

5 Conclusions

This study tackled a variant of the longest common subsequence problem which involves gap constraints for each input sequence, with applications in comparison of biological molecules and time-series analysis. We designed a learned heuristic, represented by the outputs of a neural network of predefined structure, whose weights are automatically set up by a biased random key genetic algorithm in the process of wights evolution. More advances heuristic is designed by combining the learned heuristic and the best performed hand-crafted heuristic from the literature, designing an ensemble heuristic. Further, this heuristic is incorporated into the best heuristic methodology, the iterative multi-source beam search, to guide the search. The experimental evaluation demonstrated that this approach outperformed all other competing approaches on the known synthetic problem instances from literature as well as newly introduced real problem instances that simulate energy-aware context between successive nucleotides via the data-driven gap constraints. Future work could involve the testing scenario for protein molecules, and instances with much larger sequence lengths to further asses the scalability od the designed approaches. Another task aims at learning the NN-based heuristic by some other meta-heuristic involving different fitness evaluators besides the utilized mean value while employing different stopping criteria in the training to prevent from over-fitting.

Acknowledgments. This publication is co- funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-Curie CO-FUND Postdoctoral Programme grant agreement No.101081355– SMASH and by the Republic of Slovenia and the European Union from the European Regional Development Fund. Co-funded by European Union. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

References

1. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
2. Amari, S.i.: Backpropagation and stochastic gradient descent method. Neurocomputing **5**(4-5), 185–196 (1993)
3. Bishop, C.M.: Neural networks and their applications. Review of scientific instruments **65**(6), 1803–1832 (1994)

4. Djukanovic, M., Balaban, N., Blum, C., Kartelj, A., Dzeroski, S., Zebec, Z.: On solving a generalized variable gapped longest common subsequence problem. In: Proceedings of the 20th International Conference on Computer Aided Systems Theory. Springer, 2026. Accepted
5. Djukanovic, M., Raidl, G., Blum, C.: Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)
6. Djukanović, M., Reixach, J., Nikolikj, A., Eftimov, T., Kartelj, A., Blum, C.: A learning search algorithm for the restricted longest common subsequence problem. Expert Systems with Applications **284**, 127731 (2025)
7. Gurney, K.: An introduction to neural networks. CRC press (2018)
8. Lafond, M., Lai, W., Liyanage, A., Zhu, B.: The longest subsequence-repeated subsequence problem. In: International Conference on Combinatorial Optimization and Applications. pp. 446–458. Springer (2023)
9. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
10. Lin, G., Chen, Z.Z., Jiang, T., Wen, J.: The longest common subsequence problem for sequences with nested arc annotations. Journal of Computer and System Sciences **65**(3), 465–480 (2002)
11. Mienye, I.D., Swart, T.G., Obaido, G.: Recurrent neural networks: A comprehensive review of architectures, variants, and applications. Information **15**(9), 517 (2024)
12. Peng, Y.H., Yang, C.B.: Finding the gapped longest common subsequence by incremental suffix maximum queries. Information and Computation **237**, 95–100 (2014)
13. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)
14. Pérez Cáceres, L., López-Ibáñez, M., Stützle, T.: An analysis of parameters of irace. In: European Conference on Evolutionary Computation in Combinatorial Optimization. pp. 37–48. Springer (2014)
15. Reixach, J., Blum, C., Djukanović, M., Raidl, G.R.: A neural network based guidance for a brkga: An application to the longest common square subsequence problem. In: European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar). pp. 1–15. Springer (2024)
16. SantaLucia Jr, J.: A unified view of polymer, dumbbell, and oligonucleotide dna nearest-neighbor thermodynamics. Proceedings of the National Academy of Sciences **95**(4), 1460–1465 (1998)
17. Watkins Jr, N.E., SantaLucia Jr, J.: Nearest-neighbor thermodynamics of deoxyinosine pairs in dna duplexes. Nucleic acids research **33**(19), 6258–6267 (2005)
18. Wiseman, S., Rush, A.M.: Sequence-to-sequence learning as beam-search optimization. In: Proceedings of the 2016 conference on empirical methods in natural language processing. pp. 1296–1306 (2016)
19. Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. IEEE transactions on neural networks and learning systems **33**(9), 5057–5069 (2021)